

App Dev Project Report

1. Student Details

- **Name:** Suman Ghosh
 - **Roll Number:** 24f2008859
 - **Email:** 24f2008859@ds.study.iitm.ac.in
 - **About Me:** I am a Student at IIT Madras BS Degree program with a deep interest in coding and math.
-

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

Institutes need efficient systems to manage campus recruitment activities involving companies, students, and placement drives. Currently, many institutes rely on spreadsheets, emails, or manual processes, which makes it difficult to manage company approvals, track student applications, avoid duplicate registrations, and maintain placement records.

Approach:

I built a web-based placement portal using Flask and SQLite that allows three types of users - Admin, Company, and Student — to interact based on their roles.

- Admin manages the entire portal
- Companies register, post drives and manage applications
- Students register, apply for drives and track status

I used session-based authentication for role-based access control and SQLAlchemy for database management.

3. AI/LLM Declaration

I used Claude AI as a guidance tool in this project. Claude helped me understand Flask concepts, debug errors and guide me through building features.

All the code was written by me. So, there are many bugs which I fix and I understand each concept before moving forward and I can explain every part of my code during viva.

I used it in the limit which is mentioned in the document.

Flask - Backend logic - 15%

SQLite + SQLAlchemy - Data Modeling and relationship - 15%

HTML + Jinja2 - Server-side template rendering - 5%

API - Fetching or posting data to external/internal APIs - 2%

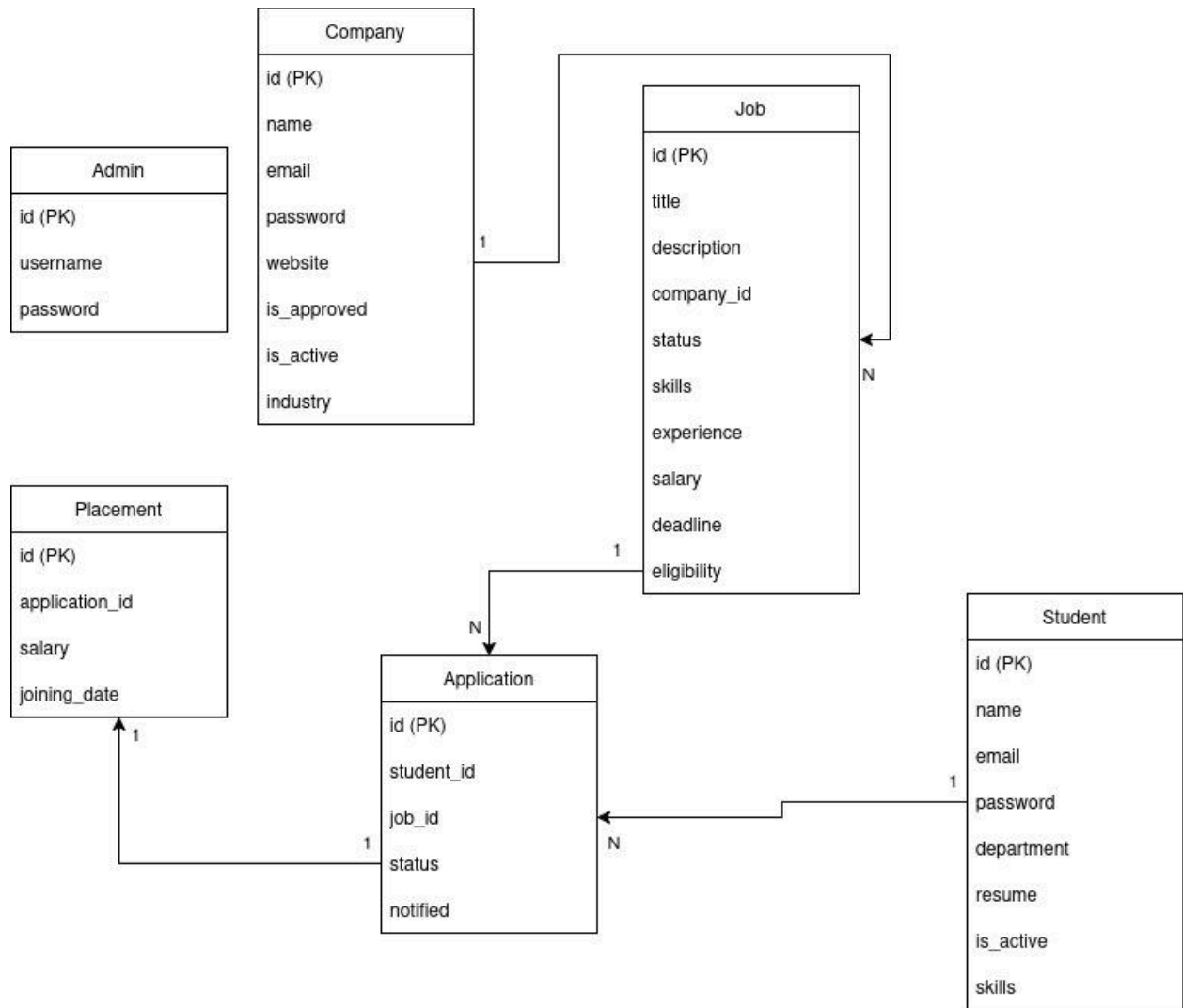
4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
HTML5	Frontend styling
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database for storing user data
Flask Sessions	Authentication

5. Database Schema / ER Diagram

Tables:

Table	Primary Key	Foreign Key
Admin	id	None
Student	id	None
Company	id	None
Job	id	company_id -> Company
Application	id	student_id -> Student, job_id -> Job
Placement	id	application_id -> Application



Relationships:

- Company has many jobs (One to Many)
 - Student has many applications (One to Many)
 - Job has many applications (One to many)
 - Application has one Placement (One to One)
 - Admin is standalone (no relationship)
-

6. API Resource Endpoints

Method	Endpoint	Description
GET	<code>/api/students</code>	Get all registered students
GET	<code>/api/companies</code>	Get all registered companies
GET	<code>/api/jobs</code>	Get all placement drives
GET	<code>/api/applications</code>	Get all applications
POST	<code>/api/jobs</code>	Create new placement drive
PUT	<code>/api/applications/<id></code>	Update application status
DELETE	<code>/api/jobs/<id></code>	Delete a placement drive

7. Architecture and Features (optional)

PlaceMe follows a Model-View-Controller(MVC) pattern:

- Model ([models.py](#)) = Database tables and relationships
- View (templates/) = HTML pages using Jinja2
- Controller ([app.py](#)) = Flask routes and business logic

Project Structure:

- PlaceMe/
 - [app.py](#) Main Flask application
 - [models.py](#) Database models
 - [README.md](#) Setup instructions
 - static/

- resumes/ Uploaded student resumes
- templates/
 - base.html Base template
 - home.html Landing page
 - login.html Login page
 - student_register.html
 - recruiter_register.html
 - admin_dashboard.html
 - student_dashboard.html
 - company_dashboard.html
 - view_applications.html
 - update_profile.html

Key Features:

Authentication:

- Session based role authentication
- Three roles: Admin, Company, Student
- Protected routes for each role

Admin Features:

- Approve/reject companies and drives
- Search and blacklist users
- View all data in one dashboard

Company Features:

- Create placement drives
- Manage applications
- View student resumes

Student Features:

- Apply for drives
- Track application status
- Receive notifications
- Upload resume

API Features:

- 4 GET endpoints
 - 1 POST endpoint
 - 1 PUT endpoint
 - 1 DELETE endpoint
-

8. Video Presentation

Demo Video presentation link:

[Demo project video](#)
